

Test Plan — Room Planner Prototype

Version: 1.0 · 2026-10-01
Companion to: Spec V2 Rev 4 · Phase 1 Contract Rev 3 · Appendix B
Home: repo root as `TESTING.md`. The spec is frozen; this plan is a living repo artifact and may grow as defects teach new cases. The Vitest matrix in Phase 1 §10 remains the canonical unit-test list; this plan makes it **executable** — fixtures, properties, gates, and CI rules.

1. Structure & Tooling

```
/
├─ src/ # engine (Phase 1 modules) + adapters
├─ tests/
│   ├─ unit/ # mirrors Phase 1 §10 matrix, one file per module
│   ├─ fixtures/ # canonical projects + expected results (§2)
│   ├─ property/ # fast-check property-based tests (§3)
│   ├─ integration/ # store ↔ commands ↔ history ↔ serialization
│   └─ perf/ # smoke benchmarks (§6)
└─ TESTING.md # this document
```

- **Runner:** Vitest. **Property tests:** fast-check.
- **Tolerance rule (binding):** all geometric assertions use `EPSILON = 1e-6 m` tolerance comparisons; linear values assert exact equality (they are quantized); rotation asserts tolerance $\leq 1e-9$ rad. A test asserting exact float equality on a geometric predicate is a defect in the test.
- **Determinism rule (binding):** no test may depend on `Date.now()`, unseeded randomness, or iteration order of unordered collections. Any randomness uses fixed seeds. This is enforced because command purity and replay determinism are contract invariants (Spec Rules 18–20).

2. Canonical Test Fixtures

Versioned JSON project files, committed with **expected validation results alongside**. They serve triple duty: engine conformance tests, serialization round-trip tests, and perf inputs (§6). Schema follows Phase 1 §3 (schemaVersion 1, vertex-ID walls).

Fixture	Content	Expected result
<code>fixtures/rect-room-basic.json</code>	4×5 m rectangular room, wall thicknesses 150/200/150/100 mm; sofa (2.0×0.9) + coffee table (1.1×0.6), both valid, door + window validly attached	<code>valid: true</code> , zero violations; round-trips byte-identically (linear values)
<code>fixtures/concave-invalid.json</code>	L-shaped room; sofa rotated 37° crossing the notch edge	<code>valid: false</code> ; exactly one hard <code>outside_room</code> violation naming the sofa; involved room IDs correct
<code>fixtures/door-swing-edge.json</code>	Door with swingAngle 75°, hingeSide 'left'; chair intersecting access zone edge within EPSILON; chair fully inside swing sector	Chair A (touching) → valid; Chair B (inside) → hard <code>door_swing</code> ; second door overlapping first → soft door-vs-door violation only

Fixture	Content	Expected result
<code>fixtures/clearance-matrix.json</code>	Sofa with hard 0.75 m front clearance; (a) chair in zone, (b) zone overlapping room boundary, (c) two clearance zones overlapping, (d) rotated sofa, side-specific policy	(a) hard <code>clearance</code> ; (b) no violation (room-boundary exemption); (c) no violation; (d) zone correctly rotated
<code>fixtures/wall-edit-cascade.json</code>	Room + 4 furniture; an EditWall command pulls the north wall inward 0.8 m	Edit commits (pipeline exception); 2 objects derived-invalid with correct violation types; fixtures clamped per A3; inverse restore returns prior geometry + validity by vertex ID
<code>fixtures/perf-100.json</code>	100 furniture instances in a large irregular room, mixed rotations/elevations	Baseline for §6 benchmarks; all-valid
<code>fixtures/quantize-edge.json</code>	Objects with values at 0.0004 / 0.0005 / 0.0006 m and corners exactly on boundary	Quantization outcomes per Phase 1 §10; boundary corners classified touching

Fixture rules:

- Fixtures are frozen: changing an expected result requires a one-line justification in the PR description (it usually signals a contract change, which must be raised per Spec §2).
- Every fixture must round-trip through `serialize.ts` with linear values exact and rotation within 1e-9 rad.
- Fixture files double as the serialization conformance suite for `schemaVersion`.

3. Property-Based Tests (geometry core)

Example tests catch known cases; property tests catch unknown ones. Minimum set:

Property	Invariant tested
Quantize idempotence	<code>∀x: quantizeLinear(quantizeLinear(x)) == quantizeLinear(x)</code> — including values within 1e-9 of midpoints and negatives (arbitrary precision floats)
Quantize bounds	<code>∀x: quantizeLinear(x) ∈ [x - 0.0005, x + 0.0005]</code> (mod EPSILON)
Collision symmetry	<code>∀ footprint pairs: collides(a,b) == collides(b,a)</code> — random rotations, sizes, elevations
Touching ≠ collision	Translating one footprint along a separating axis by exactly EPSILON from touching never flips the predicate in either direction
Containment under quantization	A valid inside placement, quantized, is never re-classified outside (corner-on-boundary → touching); random inside placements stay valid post-quantize
Escape-rule monotonicity	For the wall-edit-cascade fixture's invalid objects, random moves toward the nearest-valid target never <i>increase</i> the violation set (Phase 1 §6 escape rule)
Replay determinism	Any random command sequence (place/move/resize/rotate/undo/redo, seeded) ends in an identical serialized state — run each sequence twice, compare hashes
Inverse correctness	<code>∀ command c, state s: apply(inverse(c), apply(c, s)) == s</code> (structural equality; composite inverses in reverse order)
Polygon validity stability	Random simple polygons normalize to CCW; generated degenerate polygons (zero-area, self-intersecting, duplicate vertex) are rejected with the correct stable error code
SAT-vs-AABB consistency	Where an AABB prefilter exists, it never rejects a pair the narrow phase reports as colliding (no false broad-phase negatives)

4. Milestone Test Gates

Each milestone's definition of done is mechanical — the listed suites are green in CI.

M1 — Spatial core

- Full Phase 1 §10 unit matrix green (all 19 exit criteria covered).
- All §2 fixtures pass validation checks.
- All §3 property tests green.
- Perf smoke: fixture validation < 5 ms (§6).

M2 — First 2D editing loop (adds interaction-level tests)

- Commit pipeline: rejected commit leaves state and history untouched (deep-equal + history length unchanged).
- Invalid drag → animate-back produces **zero** history entries.
- One drag = exactly one command = exactly one history entry (spy on history.push).
- Snap ranking: deterministic tie-break verified with the §2 fixtures.
- Undo/redo through a scripted 50-step random session ends in the replayed state.

M3 — Architectural editing

- Wall-edit cascade: pre-commit live validity preview matches post-commit derived flags.
- Vertex insert/delete preserves all other segments' references (vertex-ID scheme); malformed ID reference throws `ApplyErrorCode`.
- Escape-rule tests (§3) green on real interaction commands.
- Re-validation after every wall edit is complete: every instance and fixture has a derived status.

M4 — 3D and shared interaction

- Selection sync: selecting in 2D selects in 3D by domain ID, and vice versa; occluded/hidden selected objects remain selected (Appendix B B6).
- Axis mapping round-trip: `threeToWorld(worldToThree(p)) == p` within EPSILON for random points, including elevation.
- View switch mid-drag cancels cleanly: no command, no history entry, transform restored.

M5 — Professional workflow

- Save/load: fixture files reload byte-equivalent (linear) and validation-equivalent.
- Metadata edits produce single commands; imperial formatting/parse round-trips (when imperial ships — else skipped, not stubbed).
- Export snapshot tests: schedule contains every instance with correct dimensions/vendor/sku fields from fixture metadata.

5. CI Policy

- **Every commit:** unit + fixtures + property tests. Full suite must complete < 2 min; property test case counts capped (e.g. 1000 cases/property) to keep CI fast.
- **Malformed-command tests:** every `ApplyErrorCode` has at least one test producing it.

- **Regression rule:** every user-visible defect gets a reproducing test before the fix PR merges; the test is named after the defect.
 - **Forbidden in tests:** unseeded randomness, wall-clock time, exact-equality assertions on geometric predicates (see §1 tolerance rule).
 - **Coverage target:** the `src/engine` (pure domain) directory $\geq 95\%$ line coverage; adapters and renderers have no coverage mandate — they are covered by M4/M5 integration suites instead.
-

6. Performance Smoke Tests

Not FPS budgets (unmeasurable in unit tests) — deterministic ceilings on the `perf-100.json` fixture, generous enough to avoid flaky CI, tight enough to catch regressions:

Operation	Ceiling (CI machine, generous)
Full validation pass (100 objects, one candidate)	< 5 ms
Snap candidate generation + ranking	< 2 ms
Single <code>apply</code> (composite, 10 sub-commands)	< 1 ms
Serialize + deserialize full project	< 10 ms
Fixture load + validate + replay 50-command history	< 50 ms

If a ceiling is exceeded, the fix is algorithmic (broad-phase, caching), never "raise the ceiling" — raising a ceiling is a Spec change and gets raised per Spec §2.

7. What This Plan Deliberately Does Not Cover

- Visual regression (Spec §12) — requires the renderer layer; tracked separately once M2 exists.
 - E2E browser workflows — Playwright suite defined after M2, seeded with the same fixtures.
 - Designer-usability validation — the M2 validation gate with real users, not an automated concern.
-

End of Test Plan v1